

CS-590 Database Design and Development

6-2 Assignment: Developing a Schema for a Graph Database

Malgorzata Debska

Southern New Hampshire University

Zach Probst

March 10, 2026

Developing a Schema for a Graph Database Using Neo4j

In this assignment, a relational database used by the U.S. Department of Agriculture (USDA) was analyzed to decide whether migrating the data to a graph database would provide advantages for querying connected information. Graph databases store information as nodes, relationships, and properties instead of traditional relational tables. This structure is particularly useful when working with highly connected datasets. The goal of this activity was to analyze four datasets related to food and nutrition: Food Description (FDES), Food Category Name (CTGNME), Global Product Classification Category Name (GPCNME), and Weight (WGHT). These datasets were originally stored in relational format and were transformed into nodes and relationships within a Neo4j graph database. The assignment involved finding logical entities, designing a graph schema, implementing the schema in Neo4j, performing extract–transform–load (ETL) operations on the datasets, and verifying the integration using Cypher queries.

Identifying Logical Entities

The first step in the process was showing the logical entities contained in the datasets. Each CSV file stood for a different category of information that could be converted into a node label within the graph database. The Food Description (FDES) dataset has individual food product records including brand names, descriptions, product codes, and classification of identifiers. These records are the primary entities within the graph database.

The Food Category Name (CTGNME) dataset holds information about food categories. Each entry is a category of grouping used to classify food products. The Global Product Classification Category Name (GPCNME) dataset provides standardized global classification codes used to categorize products at a broader level. The Weight (WGHT) dataset contains measurement information related to food products, including serving size, measurement description, and unit information.

Based on this analysis, four primary graph entities were found:

- Food Description (FDES)
- Food Category (CTGNME)
- Global Product Classification (GPCNME)
- Weight Measurement (WGHT)

Each entity was implemented as a node label within the Neo4j graph database.

Developing the Graph Schema

After naming the entities, the next step was to design a graph schema that defined the nodes, relationships, and properties present in the graph model. Each node type includes properties extracted from the corresponding CSV dataset.

For example:

FDES nodes have properties such as

- FoodCategoryCode (string)
- Description (string)

- ProductCode (string)
- BrandName (string)
- StartDate (date/string)
- EndDate (date/string)

CTGNME nodes have

- FoodCategoryCode (string)
- CategoryDescription (string)

GPCNME nodes have

- GPCCode (string)
- GPCDescription (string)

WGHT nodes have

- ProductCode (string)
- Amount (float)
- MeasurementDescription (string)
- UnitAmount (float)
- SequenceNumber (integer)

Relationships were defined between nodes using shared identifiers from the datasets.

The primary relationships included:

- **FDES --> CTGNME**

Food description records connect to their corresponding food category.

- **FDES --> GPCNME**

Food products may connect to global classification categories using the GPC code.

- **FDES --> WGHT**

Food products connect to weight measurement records that describe serving size and measurement information.

This graph structure allows queries to traverse relationships between products, categories, classifications, and measurement data more efficiently than traditional relational joins.

Graph Schema Diagram

A visual schema diagram was created to illustrate the node types and relationships within the graph model.

The diagram shows:

Nodes

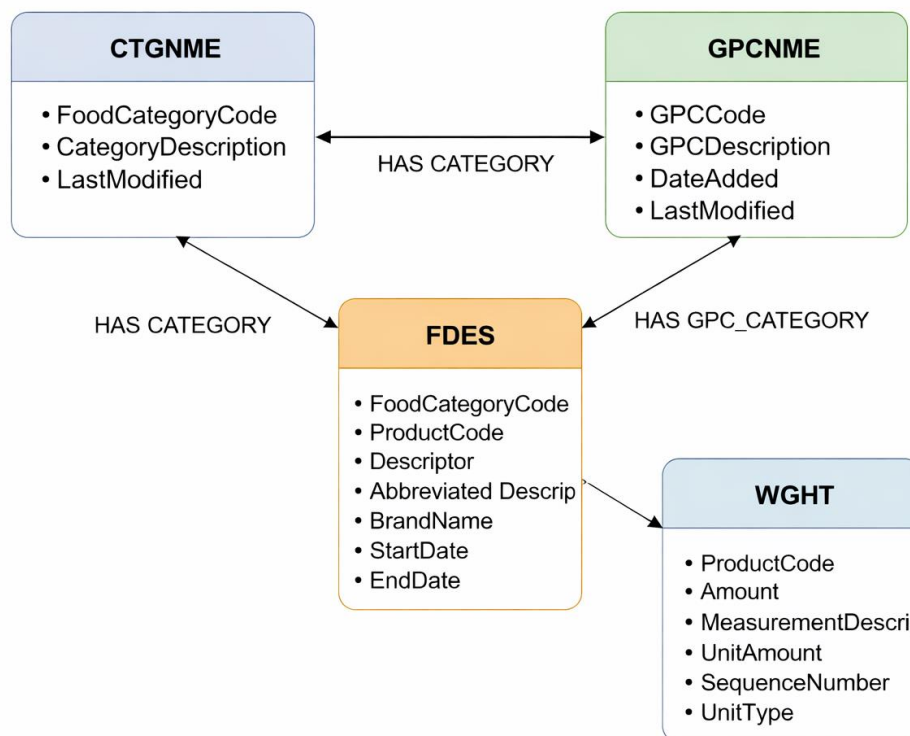
- FDES
- CTGNME
- GPCNME
- WGHT

Relationships

- HAS_CATEGORY

- HAS_GPC_CATEGORY
- HAS_WEIGHT

The Food Description node serves as the central node connecting to category, classification, and weight measurement nodes. This design allows the graph database to have real-world relationships between food products and their associated classification and measurement information.



Implementing the Graph Schema

After designing the schema, it was implemented in Neo4j using Cypher queries. The data sets were imported using the LOAD CSV WITH HEADERS command. Each dataset was stored in the Neo4j import directory and loaded into the graph database as nodes.

For example:

- The **CTGNME** dataset was loaded to create food category nodes.
- The **FDES** dataset created nodes being individual food products.
- The **GPCNME** dataset created nodes standing for global classification categories.
- The **WGHT** dataset created nodes being measurement information.

During the import process, CSV column values were mapped to node properties to ensure that the graph accurately stood for the original dataset.

Extract, Transform, and Load (ETL) Process

The ETL process played a critical role in preparing the datasets for the graph database. The extract phase involved retrieving the CSV files from the Neo4j imports directory. The transform phase mapped column names from the CSV files to node properties used within the graph schema. Data formatting adjustments were applied where necessary to ensure that the values were compatible with Neo4j property types. The load phase executed Cypher queries that created nodes and stored the data within the graph database. This process ensured that all datasets were properly integrated into the Neo4j environment.

Verifying the Data Integration

After importing the data, Cypher queries were used to verify that the nodes and properties were successfully created.

MATCH and RETURN queries were executed to confirm that:

- CTGNME nodes were successfully created
- FDES nodes were successfully created
- GPCNME nodes were successfully created
- WGHT nodes were successfully created

Additional Cypher queries were executed to create relationships between nodes based on shared identifiers.

The relationship query connecting FDES and CTGNME nodes successfully created category relationships between food products and their categories. A relationship query trying to connect FDES nodes with GPCNME nodes did not return created relationships in the results shown, saying that more validation of matching keys may be necessary. These verification queries confirmed that much of the data was successfully imported into the graph database.

To conclude, this activity proved how relational datasets can be transformed into a graph database structure using Neo4j. By naming logical entities, designing a graph schema, implementing nodes and relationships, and performing ETL operations, the relational data was

successfully converted into a graph-based model. Graph databases provide advantages when working with highly connected data because relationships are stored directly within the database structure. This enables faster traversal of related information compared to traditional relational databases that require multiple table joins. While relational databases stay effective for structured tabular data, graph databases offer significant benefits for exploring complex relationships between entities. In this case, the Neo4j implementation shows how food products, categories, and measurement information can be efficiently connected and queried within a graph database.

```
ubuntu@i-01219676b41cd8a43:~$ ls /var/lib/neo4j/import
Module_Six_CTGNME.csv  Module_Six_GPCNME.csv  Module_Six_Nut_Vals.csv
Module_Six_FDES.csv   Module_Six_Nut_Descs.csv  Module_Six_WGHT.csv
```

Screenshots of Cypher load csv for CTGNME and Match return

```
Terminal - ubuntu@i-01219676b41cd8a43: ~
File Edit View Terminal Tabs Help
cypher-shell: command not found
ubuntu@i-01219676b41cd8a43:~$ cypher-shell -u neo4j -p gosiad26
Connected to Neo4j 4.1.0 at neo4j://localhost:7687 as user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j@neo4j> LOAD CSV WITH HEADERS FROM 'file:///Module_Six_CTGNME.csv' AS row C
REATE (c:CTGNME) SET c.FoodCategoryCode = row.`Food category code`, c.CategoryD
escription = row.`Category description`, c.DateAdded = row.`Date added`, c.LastM
odified = row.`Last modified`;
0 rows available after 109 ms, consumed after another 0 ms
Added 25 nodes, Set 100 properties, Added 25 labels
neo4j@neo4j> MATCH (c:CTGNME) RETURN keys(c), c LIMIT 5;
+-----+
| keys(c) | c |
+-----+
| ["DateAdded", "LastModified", "FoodCategoryCode", "CategoryDescription"] | (:C
TGNME {LastModified: "04/09/2012", CategoryDescription: "Dairy and Egg Products"
, FoodCategoryCode: "1", DateAdded: "10/29/1997"}) |
```

```
neo4j@neo4j> MATCH (c:CTGNME) RETURN count(c);
+-----+
| count(c) |
+-----+
| 25        |
+-----+

1 row available after 223 ms, consumed after another 2 ms
neo4j@neo4j> █
```

Screenshots of Cypher load csv and Match return

```

neo4j@neo4j> LOAD CSV WITH HEADERS FROM 'file:///Module_Six_FDES.csv' AS row
CREATE (f:FDES)
SET f.FoodCategoryCode = row.`Food category code`,
f.Descriptor = row.`Descriptor`,
f.AbbreviatedDescriptor = row.`Abbreviated descriptor`,
f.CnCode = row.`Cn code`,
f.Gtin = row.`Gtin`,
f.ProductCode = row.`Product code`,
f.BrandOwnerName = row.`Brand owner name`,
f.BrandName = row.`Brand name`,
f.FNSMaterialNumber = row.`FNS Material Number`,
f.SourceCode = row.`Source code`,
f.DateAdded = row.`Date added`,
f.LastModified = row.`Last modified`,
f.DiscontinuedDate = row.`Discontinued date`,
f.FormOfFood = row.`Form of food`,
f.FdcId = row.`Fdc id`,
f.GPCProductCode = row.`Gpc product code`;
0 rows available after 654 ms, consumed after another 0 ms
Added 8976 nodes, Set 75887 properties, Added 8976 labels
neo4j@neo4j>
neo4j@neo4j>

```

```

neo4j@neo4j> MATCH (f:FDES) return count(f);
+-----+
| count(f) |
+-----+
| 8976      |
+-----+

1 row available after 44 ms, consumed after another 1 ms

```

Screenshots of Cypher load csv for GPCNME and Match return

```
neo4j@neo4j> LOAD CSV WITH HEADERS FROM 'file:///Module_Six_GPCNME.csv' AS row C
REATE (g:GPCNME) SET g.GPCCODE = row.`Gpc code`, g.GPCDescription = row.`Gpc des
cription`, g.DateAdded = row.`Date added`, g.LastModified = row.`Last modified`;

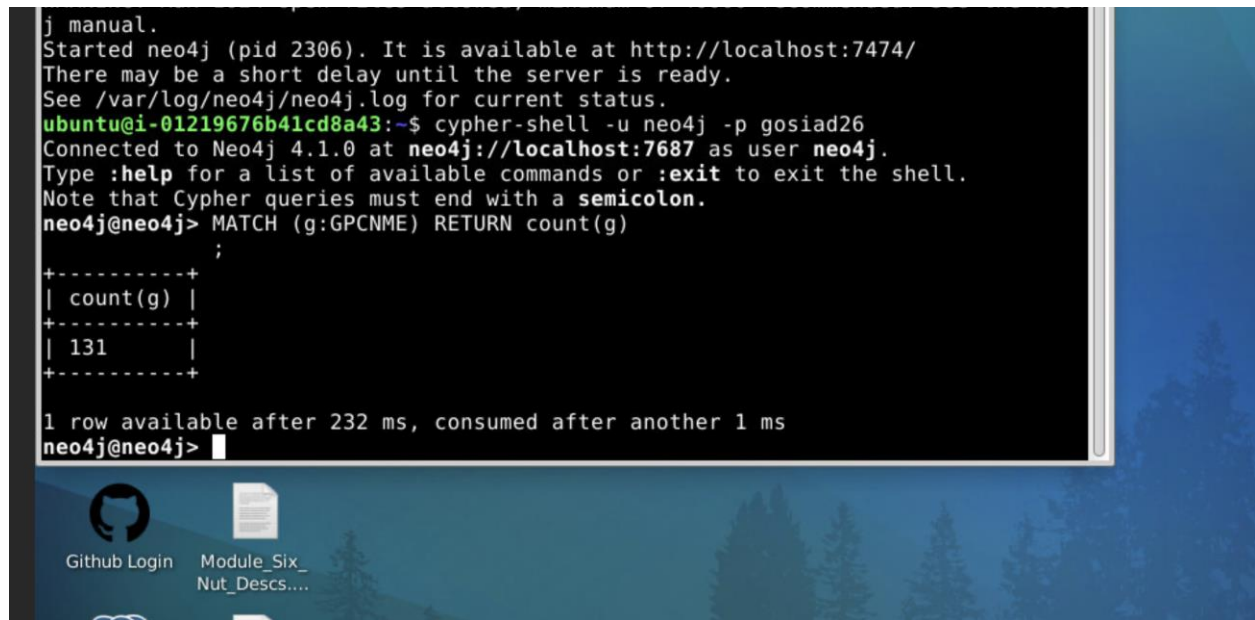
0 rows available after 93 ms, consumed after another 0 ms
Added 131 nodes, Set 524 properties, Added 131 labels
neo4j@neo4j> MATCH (g:GPCNME) RETURN keys(g), g LIMIT 5;
+-----+
+-----+
+-----+
+-----+
+-----+
```

```
File Edit View Terminal Tabs Help
+-----+
+-----+
+-----+
| ["LastModified", "GPCDescription", "GPCCODE", "DateAdded"] | (:GPCNME {LastMod
ified: "04/01/2022", GPCDescription: "Nuts/Seeds - Prepared/Processed", GPCCODE:
"50101800", DateAdded: "04/01/2022"}) |
| ["LastModified", "GPCDescription", "GPCCODE", "DateAdded"] | (:GPCNME {LastMod
ified: "04/01/2022", GPCDescription: "Fruit/Nuts/Seeds Combination", GPCCODE: "5
0101900", DateAdded: "04/01/2022"}) |
| ["LastModified", "GPCDescription", "GPCCODE", "DateAdded"] | (:GPCNME {LastMod
ified: "04/01/2022", GPCDescription: "Fruit - Prepared/Processed", GPCCODE: "501
02000", DateAdded: "04/01/2022"}) |
| ["LastModified", "GPCDescription", "GPCCODE", "DateAdded"] | (:GPCNME {LastMod
ified: "04/01/2022", GPCDescription: "Vegetables - Prepared/Processed", GPCCODE:
"50102100", DateAdded: "04/01/2022"}) |
| ["DateAdded", "LastModified", "GPCCODE", "GPCDescription"] | (:GPCNME {LastMod
ified: "04/01/2022", GPCDescription: "Fruits/Vegetables/Nuts/Seeds Variety Packs
", GPCCODE: "50102200", DateAdded: "04/01/2022"}) |
+-----+
+-----+
+-----+
5 rows available after 51 ms, consumed after another 2 ms
neo4j@neo4j> |
```

Github Login Module_Six_Nut_Descs....

```
j manual.
Started neo4j (pid 2306). It is available at http://localhost:7474/
There may be a short delay until the server is ready.
See /var/log/neo4j/neo4j.log for current status.
ubuntu@i-01219676b41cd8a43:~$ cypher-shell -u neo4j -p gosiad26
Connected to Neo4j 4.1.0 at neo4j://localhost:7687 as user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j@neo4j> MATCH (g:GPCNME) RETURN count(g)
;
+-----+
| count(g) |
+-----+
| 131      |
+-----+

1 row available after 232 ms, consumed after another 1 ms
neo4j@neo4j> 
```

A terminal window with a dark background and light text. The text shows the startup of Neo4j, a connection to the Neo4j 4.1.0 instance at localhost:7687, and a Cypher query execution. The query is 'MATCH (g:GPCNME) RETURN count(g);'. The result is a table with one row: 'count(g)' with the value '131'. Below the table, it says '1 row available after 232 ms, consumed after another 1 ms'. The prompt 'neo4j@neo4j>' is visible at the bottom of the terminal. The terminal is overlaid on a desktop background with a blue sky and trees. At the bottom of the desktop, there are icons for 'Github Login' and a file named 'Module_Six_Nut_Descs....'.

Screenshots of Cypher load csv for WGHT and Match return

```
neo4j@neo4j> LOAD CSV WITH HEADERS FROM 'file:///Module_Six_WGHT.csv' AS row CREATE (w:WGHT) SET w.CnCode = row.`Cn code`, w.SequenceNum = row.`Sequence num`, w.Amount = row.`Amount`, w.MeasureDescription = row.`Measure description`, w.UnitAmount = row.`Unit amount`, w.TypeOfUnit = row.`Type of unit`, w.SourceCode = row.`Source code`, w.DateAdded = row.`Date added`, w.LastModified = row.`Last modified`;
0 rows available after 905 ms, consumed after another 0 ms
Added 15052 nodes, Set 135468 properties, Added 15052 labels
neo4j@neo4j> █
```

```
neo4j@neo4j> MATCH (w:WGHT) RETURN count(w);
+-----+
| count(w) |
+-----+
| 15052     |
+-----+

1 row available after 130 ms, consumed after another 3 ms
neo4j@neo4j> █
```

Relationship query screenshots:

```
neo4j@neo4j> MATCH (f:FDES), (g:GPCNME) WHERE toString(f.GPCProductCode) = toString(g.GPCCode) CREATE (f)-[:HAS_GPC_CATEGORY]->(g);  
0 rows available after 101 ms, consumed after another 0 ms  
neo4j@neo4j> █
```



```
neo4j@neo4j> MATCH (f:FDES), (c:CTGNME) WHERE toString(f.FoodCategoryCode) = toString(c.FoodCategoryCode) CREATE (f)-[:IN_CATEGORY]->(c);  
0 rows available after 342 ms, consumed after another 0 ms  
Created 6848 relationships  
neo4j@neo4j> █
```